VEHICLE BRAKE CONTROL SYSTEM

AUTOSAR High-Level Design (HLD) Document

Software Architecture of BrakeControlSWC, WheelSpeedSensorSWC, BrakePressureSensorSWC, BrakeActuatorSWC, VehicleStateManagerSWC and DiagnosticManagerSWC

Document	Vehicle Brake Control System HLD
Document ID	VBCHLD-001
Revision	1.0
Status	Released for test use
Classification	Synthetic Test Document
Date	2026-10-08
Author	Synthetic Author (Architecture Team)
Reviewer	Synthetic Reviewer (Systems Team)

Notice. This document is a fictional engineering design created solely as a synthetic test fixture. It is not an AUTOSAR standard or specification, it contains no proprietary or confidential information, and it does not describe any real product.

Document Control

Table 1: Document information

Field	Value
Document	Vehicle Brake Control System HLD
Document ID	VBCHLD-001
Revision	1.0
Classification	Synthetic Test Document
System	Vehicle Brake Control System (VBCS)
Document type	High-Level Design (HLD)
Date of this revision	2026-10-08

Table 2: Revision history

Revision	Date	Author	Description of change
0.1	2026-09-01	Architecture Team	Initial draft with SW-C inventory.
0.2	2026-09-15	Architecture Team	Added interfaces, ports and signal definitions.
0.9	2026-09-29	Architecture Team	Added CAN mapping, functional flows and dependencies.
1.0	2026-10-08	Architecture Team	Added diagnostics and error handling; released for test use.

Table of Contents

1. Introduction . . . 5

1.1 Purpose . . . 5

1.2 Scope . . . 5

1.3 Definitions and Abbreviations . . . 5

1.4 Naming Conventions . . . 5

1.5 Referenced Documents . . . 6

2. System Overview . . . 6

2.1 System Description . . . 6

2.2 ECU Deployment . . . 6

2.3 Operating Modes . . . 6

2.4 Architectural Requirements . . . 6

3. AUTOSAR Architecture . . . 7

3.1 Software Component Architecture . . . 7

3.2 BrakeControlSWC . . . 7

3.3 WheelSpeedSensorSWC . . . 8

3.4 BrakePressureSensorSWC . . . 9

3.5 BrakeActuatorSWC . . . 9

3.6 VehicleStateManagerSWC . . . 10

3.7 DiagnosticManagerSWC . . . 11

4. Interfaces . . . 12

4.1 Interface Overview . . . 12

4.2 Sender-Receiver Interfaces . . . 12

4.3 Client-Server Interfaces . . . 12

4.4 Client-Server Operation Signatures . . . 13
4.5 Data Types . . . 13
4.6 Interface Classification Summary . . . 13
5. Ports . . . 13
5.1 Port Concepts . . . 13
5.2 Port Definitions . . . 14
5.3 Port Exposure Summary . . . 14
6. Signals . . . 14
6.1 Signal Overview . . . 14
6.2 Signal Definitions . . . 15
6.3 Signal-to-Interface Mapping . . . 15
6.4 Signal Producers and Consumers . . . 15
7. CAN Communication . . . 16
7.1 Network Description . . . 16
7.2 CAN Message Mapping . . . 16
7.3 Signal Layout . . . 17
7.4 Timing and Protection . . . 17
8. Functional Flows . . . 18
8.1 Flow Overview . . . 18
8.2 FF-01 Primary Braking Flow . . . 18
8.3 FF-02 Wheel-Speed to Brake-Command Flow . . . 18
8.4 FF-03 Brake-Pressure to Brake-Command Flow . . . 19
8.5 FF-04 Vehicle Speed Estimation Flow . . . 19
8.6 FF-05 ABS Status Flow . . . 19
8.7 FF-06 Brake Temperature and Degraded Mode Flow . . . 20
8.8 FF-07 Diagnostic Event Reporting Flow . . . 20
8.9 FF-08 Actuator Service Flow . . . 20
9. SW-C Dependencies . . . 21
9.1 Dependency Table . . . 21
9.2 Dependencies between BrakeControlSWC and BrakeActuatorSWC . . . 21
9.3 Dependency Matrix . . . 22
10. Diagnostics . . . 22
10.1 Diagnostic Architecture . . . 22
10.2 Diagnostic Events . . . 22
10.3 Freeze-Frame Data . . . 23
10.4 Diagnostic Self-Test . . . 23
11. Error Handling . . . 23
11.1 Error Handling Strategy . . . 23
11.2 Error Table . . . 23
11.3 Degraded Mode Behavior . . . 24

12. Architecture Summary 24

12.1 Summary of Architecture Elements 24

12.2 Key Architectural Decisions 24

12.3 Open Items 24

1. Introduction

1.1 Purpose

This High-Level Design (HLD) document, **VBCHLD-001** revision 1.0, describes the AUTOSAR software architecture of the **Vehicle Brake Control System** (VBCS). VBCS is an imaginary automotive braking-control system. The document defines the application software components (SW-Cs), their interfaces, ports and signals, the CAN communication, the functional flows between SW-Cs, the SW-C dependencies, the diagnostics concept and the error handling.

The document is a synthetic engineering design. It is intended as a test fixture for tools that ingest, parse and query AUTOSAR design documents.

1.2 Scope

The scope of this document is the software architecture at SW-C level for six SW-Cs: BrakeControlSWC, WheelSpeedSensorSWC, BrakePressureSensorSWC, BrakeActuatorSWC, VehicleStateManagerSWC and DiagnosticManagerSWC. The document covers application-layer design and the communication of these SW-Cs. The following topics are outside the scope of this revision: hardware electrical design, detailed control algorithm tuning, final calibration values, functional safety work products and the content of the AUTOSAR standard itself.

1.3 Definitions and Abbreviations

Table 3: Abbreviations used in this document

Term	Meaning
ABS	Anti-lock Braking System
AUTOSAR	Automotive Open System Architecture
BAM	Brake Actuator Module (ECU hosting BrakeActuatorSWC)
BCU	Brake Control Unit (ECU hosting BrakeControlSWC, VehicleStateManagerSWC and DiagnosticManagerSWC)
BSW	Basic Software
CAN	Controller Area Network
CRC	Cyclic Redundancy Check
DLC	Data Length Code of a CAN frame
Dem	Diagnostic Event Manager (BSW module)
DTC	Diagnostic Trouble Code (fictional identifiers are used in this document)
E2E	End-to-end protection of CAN messages using a counter and CRC
ECU	Electronic Control Unit
HLD	High-Level Design
RTE	Runtime Environment
SIE	Sensor Interface ECU (hosts WheelSpeedSensorSWC and BrakePressureSensorSWC)
SW-C	Software Component
VBCS	Vehicle Brake Control System

1.4 Naming Conventions

Table 4: Naming conventions

Element	Convention	Example
Software component	Name ends with SWC	BrakeControlSWC
Provided port	Prefix PP_	PP_BrakeCommand
Required port	Prefix RP_	RP_WheelSpeed
Port interface	Name ends with If	WheelSpeedIf
Signal	Name in CamelCase with optional wheel position suffix	WheelSpeed_FL, BrakePressure
CAN message	UPPER_CASE with two-digit version suffix	BRAKE_CMD_01
Diagnostic event	Prefix EVT_	EVT_BPS_OutOfRange
Runnable	Component or function name with trigger period	BrakeControl_Run10ms

1.5 Referenced Documents

This document is self-contained. No external documents are required to read it. Related fictional document identifiers VBCSYS-001 (System Requirements) and VBCTST-001 (Test Specification) are mentioned only as placeholders and are not part of this test fixture.

2. System Overview

2.1 System Description

The Vehicle Brake Control System controls hydraulic brake actuation. The driver brake demand is measured as the hydraulic **BrakePressure**. **BrakeControlSWC** combines this demand with the four wheel speeds and the estimated **VehicleSpeed** to calculate the **BrakeCommand** sent to the brake actuator. When wheel slip is detected, the controller performs ABS modulation and signals **ABS_Active**. **DiagnosticManagerSWC** supervises the system and can request degraded mode through **DegradedModeRequest**.

2.2 ECU Deployment

VBCS is distributed over three ECUs connected by one CAN network named VBC_CAN_A. Table 5 lists the deployment of the six SW-Cs.

Table 5: ECU deployment of SW-Cs

ECU	Full name	Hosted SW-Cs	CAN node
SIE	Sensor Interface ECU	WheelSpeedSensorSWC BrakePressureSensorSWC	VBC_CAN_A
BCU	Brake Control Unit	BrakeControlSWC VehicleStateManagerSWC DiagnosticManagerSWC	VBC_CAN_A
BAM	Brake Actuator Module	BrakeActuatorSWC	VBC_CAN_A

2.3 Operating Modes

Table 6: System operating modes

Mode	Entry condition	Behavior
Initialization	Power-up of the BCU	All Init runnables execute once; outputs hold safe default values.
Normal	Initialization complete and DegradedModeRequest equals FALSE	Full ABS and brake-fade compensation are available.
Degraded	DegradedModeRequest equals TRUE	BrakeControlSWC limits BrakeCommand, requests an actuator pressure limit of 120 bar and may disable ABS.

The vehicle mode signal **VehicleMode** uses the data type VehicleModeType with the values VEH_MODE_OFF (0), VEH_MODE_STANDBY (1), VEH_MODE_DRIVE (2) and VEH_MODE_SERVICE (3).

2.4 Architectural Requirements

Table 7: High-level architectural requirements

ID	Requirement
VBC-HLR-001	BrakeControlSWC shall execute BrakeControl_Run10ms with a period of 10 ms.
VBC-HLR-002	WheelSpeedSensorSWC shall provide WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR at least every 10 ms.
VBC-HLR-003	BrakePressureSensorSWC shall provide BrakePressure in the range 0 to 250 bar.
VBC-HLR-004	BrakeControlSWC shall provide BrakeCommand to BrakeActuatorSWC through BrakeCommandIf.
VBC-HLR-005	VehicleStateManagerSWC shall provide VehicleSpeed and VehicleMode through VehicleStateIf.
VBC-HLR-006	All SW-Cs other than DiagnosticManagerSWC shall report faults through DiagnosticEventIf.
VBC-HLR-007	All CAN messages shall be protected with a 4-bit counter and an 8-bit CRC.
VBC-HLR-008	DiagnosticManagerSWC shall set DegradedModeRequest to TRUE when BrakeTemperature exceeds 600 degC for 2 s.
VBC-HLR-009	BrakeActuatorSWC shall enforce the pressure limit set through SetActuatorPressureLimit.

3. AUTOSAR Architecture

3.1 Software Component Architecture

VBCS consists of six SW-Cs. Three are Application SW-Cs (BrakeControlSWC, VehicleStateManagerSWC and DiagnosticManagerSWC); the others are Sensor/Actuator SW-Cs that abstract the sensors and the actuator. SW-Cs communicate exclusively through ports that reference port interfaces. The Runtime Environment (RTE) routes data between SW-Cs on the same ECU. Between ECUs, the RTE uses the Com, PduR and CanIf modules of the basic software to exchange the CAN messages described in Section 7.

Table 8: SW-C inventory

ID	SW-C name	Component type	ECU	Provided ports	Required ports	Description
SWC-01	BrakeControlSWC	Application SW-C	BCU	3	7	Calculates brake actuator commands based on wheel-speed and brake-pressure information.
SWC-02	WheelSpeedSensorSWC	Sensor/Actuator SW-C (Sensor SW-C)	SIE	1	1	Acquires the four wheel speed measurements, validates them and provides WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR.
SWC-03	BrakePressureSensorSWC	Sensor/Actuator SW-C (Sensor SW-C)	SIE	1	1	Samples the hydraulic brake pressure transducer and provides the validated BrakePressure signal.
SWC-04	BrakeActuatorSWC	Sensor/Actuator SW-C (Actuator SW-C)	BAM	2	2	Converts BrakeCommand into hydraulic actuation, measures BrakeTemperature and offers actuator services.
SWC-05	VehicleStateManagerSWC	Application SW-C	BCU	1	3	Estimates VehicleSpeed from the wheel speeds and derives the vehicle mode.
SWC-06	DiagnosticManagerSWC	Application SW-C	BCU	2	5	Collects diagnostic events, monitors brake system health and requests degraded mode when required.

In total VBCS contains 6 SW-Cs, 10 port interfaces (7 Sender-Receiver and 3 Client-Server), 29 ports (10 provided and 19 required) and 11 signals.

Table 9: Basic software modules used

ECU	BSW modules	Purpose
SIE	Com, PduR, CanIf, CanDrv	Transmit WHEEL_SPEED_01 and BRAKE_PRESSURE_01.
BCU	Com, PduR, CanIf, CanDrv, Dem	Receive sensor and actuator messages; transmit BRAKE_CMD_01, ABS_STATUS_01 and VEHICLE_STATE_01; store diagnostic events.
BAM	Com, PduR, CanIf, CanDrv	Receive BRAKE_CMD_01; transmit BRAKE_TEMP_01.

3.2 BrakeControlSWC

BrakeControlSWC (SWC-01) is a **Application SW-C** deployed on the BCU. Calculates brake actuator commands based on wheel-speed and brake-pressure information.

Table 10: BrakeControlSWC specification

Attribute	Value
SW-C name	BrakeControlSWC
Component type	Application SW-C
Description	Calculates brake actuator commands based on wheel-speed and brake-pressure information.
Responsibilities	1. Calculate BrakeCommand from WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL, WheelSpeed_RR, BrakePressure and VehicleSpeed. 2. Detect wheel slip and perform ABS modulation; publish ABS_Active. 3. Apply brake-fade compensation using BrakeTemperature. 4. Honour DegradedModeRequest by limiting BrakeCommand and the actuator pressure limit. 5. Query actuator state with GetActuatorStatus and limit pressure with SetActuatorPressureLimit. 6. Report control-level faults through ReportDiagEvent and expose calibration data through BrakeControlInfoIf.
Required interfaces	WheelSpeedIf, BrakePressureIf, VehicleStateIf, BrakeTemperatureIf, DiagStatusIf, BrakeActuatorServiceIf, DiagnosticEventIf
Provided interfaces	BrakeCommandIf, ABSStatusIf, BrakeControlInfoIf
Required ports	RP_WheelSpeed, RP_BrakePressure, RP_VehicleState, RP_BrakeTemperature, RP_DiagStatus, RP_BrakeActuatorService, RP_DiagEvent
Provided ports	PP_BrakeCommand, PP_ABSStatus, PP_BrakeControlInfo

Ports exposed by BrakeControlSWC. BrakeControlSWC exposes 3 provided ports: PP_BrakeCommand (Sender, interface BrakeCommandIf), PP_ABSStatus (Sender, interface ABSStatusIf), PP_BrakeControlInfo (Server, interface BrakeControlInfoIf). It exposes 7 required ports: RP_WheelSpeed (Receiver, interface WheelSpeedIf), RP_BrakePressure (Receiver, interface BrakePressureIf), RP_VehicleState (Receiver, interface VehicleStateIf), RP_BrakeTemperature (Receiver, interface BrakeTemperatureIf), RP_DiagStatus (Receiver, interface DiagStatusIf), RP_BrakeActuatorService (Client, interface BrakeActuatorServiceIf), RP_DiagEvent (Client, interface DiagnosticEventIf).

Interfaces used by BrakeControlSWC. BrakeControlSWC uses 10 interfaces: WheelSpeedIf, BrakePressureIf, BrakeCommandIf, ABSStatusIf, VehicleStateIf, BrakeTemperatureIf, DiagStatusIf, DiagnosticEventIf, BrakeActuatorServiceIf, BrakeControlInfoIf.

BrakeControlSWC is the central application component of the Vehicle Brake Control System. It runs on the Brake Control Unit (BCU) and executes its main runnable BrakeControl_Run10ms every 10 ms. The runnable reads the four wheel speeds through RP_WheelSpeed, the driver brake demand through RP_BrakePressure, the reference speed through RP_VehicleState and the disc temperature through RP_BrakeTemperature.

The calculated BrakeCommand is written to PP_BrakeCommand and transmitted to the Brake Actuator Module in the CAN message BRAKE_CMD_01. When the slip of any wheel exceeds the active slip threshold of 15 percent, ABS modulation is started and ABS_Active is set to TRUE on PP_ABSStatus.

BrakeControlSWC is a client of BrakeActuatorServiceIf (through RP_BrakeActuatorService) and of DiagnosticEventIf (through RP_DiagEvent). It is a server of BrakeControlInfoIf (through PP_BrakeControlInfo), which DiagnosticManagerSWC uses to collect calibration data for freeze frames.

Table 11: Runnables of BrakeControlSWC

Runnable	Trigger	Period	Description
BrakeControl_Init	InitEvent	Once at start-up	Initializes controller state and loads the active calibration set.
BrakeControl_Run10ms	TimingEvent	10 ms	Reads all inputs, computes wheel slip, calculates BrakeCommand and writes outputs.
BrakeControl_OnInfoRequest	OperationInvokedEvent (PP_BrakeControlInfo)	On demand	Serves GetCalibrationSetId and GetSlipThreshold.

3.3 WheelSpeedSensorSWC

WheelSpeedSensorSWC (SWC-02) is a **Sensor/Actuator SW-C (Sensor SW-C)** deployed on the SIE. Acquires the four wheel speed measurements, validates them and provides WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR.

Table 12: WheelSpeedSensorSWC specification

Attribute	Value
SW-C name	WheelSpeedSensorSWC
Component type	Sensor/Actuator SW-C (Sensor SW-C)
Description	Acquires the four wheel speed measurements, validates them and provides WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR.
Responsibilities	1. Acquire raw wheel pulse information for the front-left, front-right, rear-left and rear-right wheels. 2. Convert the raw values to float32 wheel speeds in km/h. 3. Perform range and gradient plausibility checks on each wheel speed. 4. Provide WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR through PP_WheelSpeed. 5. Report wheel speed sensor faults through ReportDiagEvent.
Required interfaces	DiagnosticEventIf
Provided interfaces	WheelSpeedIf
Required ports	RP_DiagEvent
Provided ports	PP_WheelSpeed

Ports exposed by WheelSpeedSensorSWC. WheelSpeedSensorSWC exposes 1 provided ports: PP_WheelSpeed (Sender, interface WheelSpeedIf). It exposes 1 required ports: RP_DiagEvent (Client, interface DiagnosticEventIf).

Interfaces used by WheelSpeedSensorSWC. WheelSpeedSensorSWC uses 2 interfaces: WheelSpeedIf, DiagnosticEventIf.

WheelSpeedSensorSWC runs on the Sensor Interface ECU (SIE). It transmits four signals: WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR. All four are carried by WheelSpeedIf and are mapped to the CAN message WHEEL_SPEED_01 (CAN ID 0x1A0), which is transmitted every 10 ms.

The plausibility check rejects any wheel speed outside the range 0 to 350 km/h and any speed change larger than 30 km/h within one 10 ms cycle. A rejected value is reported as diagnostic event EVT_WSS_FL_Implausible, EVT_WSS_FR_Implausible, EVT_WSS_RL_Implausible or EVT_WSS_RR_Implausible respectively.

Table 13: Runnables of WheelSpeedSensorSWC

Runnable	Trigger	Period	Description
WheelSpeed_Init	InitEvent	Once at start-up	Initializes the sensor filters.
WheelSpeed_Run10ms	TimingEvent	10 ms	Acquires, filters and validates the wheel speeds and writes them to PP_WheelSpeed.

3.4 BrakePressureSensorSWC

BrakePressureSensorSWC (SWC-03) is a **Sensor/Actuator SW-C (Sensor SW-C)** deployed on the SIE. Samples the hydraulic brake pressure transducer and provides the validated BrakePressure signal.

Table 14: BrakePressureSensorSWC specification

Attribute	Value
SW-C name	BrakePressureSensorSWC
Component type	Sensor/Actuator SW-C (Sensor SW-C)
Description	Samples the hydraulic brake pressure transducer and provides the validated BrakePressure signal.
Responsibilities	1. Sample the brake pressure transducer every 10 ms. 2. Filter the measurement and convert it to float32 bar. 3. Check the measurement against the valid range of 0 to 250 bar. 4. Provide BrakePressure through PP_BrakePressure. 5. Report out-of-range conditions through ReportDiagEvent.
Required interfaces	DiagnosticEventIf
Provided interfaces	BrakePressureIf
Required ports	RP_DiagEvent
Provided ports	PP_BrakePressure

Ports exposed by BrakePressureSensorSWC. BrakePressureSensorSWC exposes 1 provided ports: PP_BrakePressure (Sender, interface BrakePressureIf). It exposes 1 required ports: RP_DiagEvent (Client, interface DiagnosticEventIf).

Interfaces used by BrakePressureSensorSWC. BrakePressureSensorSWC uses 2 interfaces: BrakePressureIf, DiagnosticEventIf.

BrakePressureSensorSWC runs on the Sensor Interface ECU (SIE) and transmits one signal: BrakePressure. The signal is carried by BrakePressureIf and mapped to the CAN message BRAKE_PRESSURE_01 (CAN ID 0x1B0). The only SW-C that receives BrakePressure is BrakeControlSWC, through its required port RP_BrakePressure.

A measurement outside 0 to 250 bar is replaced by the last valid value for at most two cycles and is reported as diagnostic event EVT_BPS_OutOfRange.

Table 15: Runnables of BrakePressureSensorSWC

Runnable	Trigger	Period	Description
BrakePressure_Init	InitEvent	Once at start-up	Initializes the transducer filter.
BrakePressure_Run10ms	TimingEvent	10 ms	Samples, filters and validates BrakePressure and writes it to PP_BrakePressure.

3.5 BrakeActuatorSWC

BrakeActuatorSWC (SWC-04) is a **Sensor/Actuator SW-C (Actuator SW-C)** deployed on the BAM. Converts BrakeCommand into hydraulic actuation, measures BrakeTemperature and offers actuator services.

Table 16: BrakeActuatorSWC specification

Attribute	Value
SW-C name	BrakeActuatorSWC
Component type	Sensor/Actuator SW-C (Actuator SW-C)
Description	Converts BrakeCommand into hydraulic actuation, measures BrakeTemperature and offers actuator services.
Responsibilities	1. Receive BrakeCommand through RP_BrakeCommand and translate it into valve and pump drive values. 2. Enforce the active actuator pressure limit. 3. Measure the brake disc temperature and provide BrakeTemperature through PP_BrakeTemperature. 4. Serve GetActuatorStatus, SetActuatorPressureLimit and RunActuatorSelfTest on PP_BrakeActuatorService. 5. Detect a missing BrakeCommand and report faults through ReportDiagEvent.
Required interfaces	BrakeCommandIf, DiagnosticEventIf
Provided interfaces	BrakeTemperatureIf, BrakeActuatorServiceIf
Required ports	RP_BrakeCommand, RP_DiagEvent
Provided ports	PP_BrakeTemperature, PP_BrakeActuatorService

Ports exposed by BrakeActuatorSWC. BrakeActuatorSWC exposes 2 provided ports: PP_BrakeTemperature (Sender, interface BrakeTemperatureIf), PP_BrakeActuatorService (Server, interface BrakeActuatorServiceIf). It exposes 2 required ports: RP_BrakeCommand (Receiver, interface BrakeCommandIf), RP_DiagEvent (Client, interface DiagnosticEventIf).

Interfaces used by BrakeActuatorSWC. BrakeActuatorSWC uses 4 interfaces: BrakeCommandIf, BrakeTemperatureIf, DiagnosticEventIf, BrakeActuatorServiceIf.

BrakeActuatorSWC runs on the Brake Actuator Module (BAM). It receives BrakeCommand from BrakeControlSWC and provides BrakeTemperature to BrakeControlSWC and DiagnosticManagerSWC. BrakeTemperature is mapped to the CAN message BRAKE_TEMP_01 (CAN ID 0x1C0), transmitted every 50 ms.

BrakeActuatorSWC is the server of BrakeActuatorServiceIf. Its clients are BrakeControlSWC (GetActuatorStatus and SetActuatorPressureLimit) and DiagnosticManagerSWC (GetActuatorStatus and RunActuatorSelfTest).

Table 17: Runnables of BrakeActuatorSWC

Runnable	Trigger	Period	Description
BrakeActuator_Init	InitEvent	Once at start-up	Initializes valve drivers and sets the default pressure limit of 200 bar.
BrakeActuator_Run10ms	TimingEvent	10 ms	Reads BrakeCommand and drives the hydraulic actuator.
BrakeActuator_TempRun50ms	TimingEvent	50 ms	Measures and publishes BrakeTemperature.
BrakeActuator_OnServiceRequest	OperationInvokedEvent (PP_BrakeActuatorService)	On demand	Serves the actuator service operations.

3.6 VehicleStateManagerSWC

VehicleStateManagerSWC (SWC-05) is a **Application SW-C** deployed on the BCU. Estimates VehicleSpeed from the wheel speeds and derives the vehicle mode.

Table 18: VehicleStateManagerSWC specification

Attribute	Value
SW-C name	VehicleStateManagerSWC
Component type	Application SW-C
Description	Estimates VehicleSpeed from the wheel speeds and derives the vehicle mode.
Responsibilities	1. Receive the four wheel speeds through RP_WheelSpeed. 2. Estimate VehicleSpeed (reference speed) in km/h. 3. Receive ABS_Active and hold the reference speed estimate during ABS modulation. 4. Derive VehicleMode (VEH_MODE_OFF, VEH_MODE_STANDBY, VEH_MODE_DRIVE, VEH_MODE_SERVICE). 5. Provide VehicleSpeed and VehicleMode through PP_VehicleState and report plausibility faults.
Required interfaces	WheelSpeedIf, ABSStatusIf, DiagnosticEventIf
Provided interfaces	VehicleStateIf
Required ports	RP_WheelSpeed, RP_ABSStatus, RP_DiagEvent
Provided ports	PP_VehicleState

Ports exposed by VehicleStateManagerSWC. VehicleStateManagerSWC exposes 1 provided ports: PP_VehicleState (Sender, interface VehicleStateIf). It exposes 3 required ports: RP_WheelSpeed (Receiver,

interface WheelSpeedIf), RP_ABSStatus (Receiver, interface ABSStatusIf), RP_DiagEvent (Client, interface DiagnosticEventIf).

Interfaces used by VehicleStateManagerSWC. VehicleStateManagerSWC uses 4 interfaces: WheelSpeedIf, ABSStatusIf, VehicleStateIf, DiagnosticEventIf.

VehicleStateManagerSWC runs on the Brake Control Unit (BCU). It estimates VehicleSpeed as the median of the two highest valid wheel speeds while ABS_Active is TRUE, and as the mean of all valid wheel speeds otherwise. VehicleSpeed and VehicleMode are provided on PP_VehicleState and mapped to the CAN message VEHICLE_STATE_01 (CAN ID 0x3A0).

Table 19: Runnables of VehicleStateManagerSWC

Runnable	Trigger	Period	Description
VehicleState_Init	InitEvent	Once at start-up	Initializes the speed estimator.
VehicleState_Run20ms	TimingEvent	20 ms	Estimates VehicleSpeed, derives VehicleMode and writes PP_VehicleState.

3.7 DiagnosticManagerSWC

DiagnosticManagerSWC (SWC-06) is a **Application SW-C** deployed on the BCU. Collects diagnostic events, monitors brake system health and requests degraded mode when required.

Table 20: DiagnosticManagerSWC specification

Attribute	Value
SW-C name	DiagnosticManagerSWC
Component type	Application SW-C
Description	Collects diagnostic events, monitors brake system health and requests degraded mode when required.
Responsibilities	1. Serve ReportDiagEvent and ClearDiagEvent on PP_DiagEvent. 2. Debounce reported events and forward them to the Diagnostic Event Manager (Dem) in the basic software. 3. Monitor BrakeTemperature and the duration of ABS_Active. 4. Evaluate system health and set DegradedModeRequest on PP_DiagStatus. 5. Start actuator self-tests with RunActuatorSelfTest when VehicleMode is VEH_MODE_STANDBY. 6. Collect freeze-frame data using GetCalibrationSetId, GetActuatorStatus, VehicleSpeed, VehicleMode, BrakeTemperature and ABS_Active.
Required interfaces	BrakeControlInfoIf, BrakeActuatorServiceIf, ABSStatusIf, VehicleStateIf, BrakeTemperatureIf
Provided interfaces	DiagnosticEventIf, DiagStatusIf
Required ports	RP_BrakeControlInfo, RP_BrakeActuatorService, RP_ABSStatus, RP_VehicleState, RP_BrakeTemperature
Provided ports	PP_DiagEvent, PP_DiagStatus

Ports exposed by DiagnosticManagerSWC. DiagnosticManagerSWC exposes 2 provided ports: PP_DiagEvent (Server, interface DiagnosticEventIf), PP_DiagStatus (Sender, interface DiagStatusIf). It exposes 5 required ports: RP_BrakeControlInfo (Client, interface BrakeControlInfoIf), RP_BrakeActuatorService (Client, interface BrakeActuatorServiceIf), RP_ABSStatus (Receiver, interface ABSStatusIf), RP_VehicleState (Receiver, interface VehicleStateIf), RP_BrakeTemperature (Receiver, interface BrakeTemperatureIf).

Interfaces used by DiagnosticManagerSWC. DiagnosticManagerSWC uses 7 interfaces: ABSStatusIf, VehicleStateIf, BrakeTemperatureIf, DiagStatusIf, DiagnosticEventIf, BrakeActuatorServiceIf, BrakeControlInfoIf.

DiagnosticManagerSWC runs on the Brake Control Unit (BCU). It is the only server of DiagnosticEventIf; all other five SW-Cs are clients through their RP_DiagEvent ports. It provides DiagStatusIf to BrakeControlSWC, and it is a client of BrakeActuatorServiceIf and BrakeControlInfoIf.

Table 21: Runnables of DiagnosticManagerSWC

Runnable	Trigger	Period	Description
DiagMgr_Init	InitEvent	Once at start-up	Initializes debounce counters.
DiagMgr_Run20ms	TimingEvent	20 ms	Monitors inputs, evaluates system health and writes DegradedModeRequest.
DiagMgr_OnReportEvent	OperationInvokedEvent (ReportDiagEvent)	On demand	Receives event reports from client SW-Cs.
DiagMgr_OnClearEvent	OperationInvokedEvent (ClearDiagEvent)	On demand	Clears event status.

4. Interfaces

4.1 Interface Overview

VBCS defines ten port interfaces. Seven are **Sender-Receiver** interfaces: WheelSpeedIf, BrakePressureIf, BrakeCommandIf, ABSStatusIf, VehicleStateIf, BrakeTemperatureIf and DiagStatusIf. Three are **Client-Server** interfaces: DiagnosticEventIf, BrakeActuatorServiceIf and BrakeControlInfoIf.

4.2 Sender-Receiver Interfaces

Table 22: Sender-Receiver interface definitions

ID	Interface name	Interface type	Data elements	Data types	Description
IF-01	WheelSpeedIf	Sender-Receiver	WheelSpeed_FL WheelSpeed_FR WheelSpeed_RL WheelSpeed_RR	float32 float32 float32 float32	Carries the four individual wheel speeds (front-left, front-right, rear-left, rear-right) measured by the wheel speed sensors.
IF-02	BrakePressureIf	Sender-Receiver	BrakePressure	float32	Carries the hydraulic brake pressure that represents the driver brake demand.
IF-03	BrakeCommandIf	Sender-Receiver	BrakeCommand	float32	Carries the brake actuator command (0 to 100 percent of maximum actuation) calculated by the brake controller.
IF-04	ABSStatusIf	Sender-Receiver	ABS_Active	boolean	Indicates whether anti-lock braking (ABS) modulation is currently active.
IF-05	VehicleStateIf	Sender-Receiver	VehicleSpeed VehicleMode	float32 VehicleModeType (uint8)	Carries the estimated vehicle reference speed and the current vehicle mode.
IF-06	BrakeTemperatureIf	Sender-Receiver	BrakeTemperature	float32	Carries the measured brake disc temperature reported by the actuator module.
IF-07	DiagStatusIf	Sender-Receiver	DegradedModeRequest	boolean	Carries the request from the diagnostic manager to operate the brake controller in degraded mode.

4.3 Client-Server Interfaces

Table 23: Client-Server interface definitions

ID	Interface name	Interface type	Operations	Return data types	Description
IF-08	DiagnosticEventIf	Client-Server	ReportDiagEvent ClearDiagEvent	Std_ReturnType Std_ReturnType	Server operations used by SW-Cs to report the pass or fail status of a diagnostic event and to clear an event.
IF-09	BrakeActuatorServiceIf	Client-Server	GetActuatorStatus SetActuatorPressureLimit RunActuatorSelfTest	Std_ReturnType Std_ReturnType Std_ReturnType	Server operations offered by the brake actuator to query status, limit the maximum pressure and start a self-test.
IF-10	BrakeControlInfoIf	Client-Server	GetCalibrationSetId GetSlipThreshold	Std_ReturnType Std_ReturnType	Server operations offered by the brake controller to expose the active calibration set and the slip threshold.

4.4 Client-Server Operation Signatures

Table 24: Operations of the Client-Server interfaces

Interface	Operation	Arguments (direction, data type, name)	Return type	Description
DiagnosticEventIf	ReportDiagEvent	in uint16 EventId; in EventStatusType EventStatus	Std_ReturnType	Reports PASSED or FAILED for the given diagnostic event.
DiagnosticEventIf	ClearDiagEvent	in uint16 EventId	Std_ReturnType	Clears the stored status of the given diagnostic event.
BrakeActuatorServiceIf	GetActuatorStatus	out ActuatorStatusType ActuatorStatus	Std_ReturnType	Returns the current actuator status (ACT_OK, ACT_BUSY, ACT_DEGRADED or ACT_FAULT).
BrakeActuatorServiceIf	SetActuatorPressureLimit	in float32 LimitBar	Std_ReturnType	Sets the maximum hydraulic pressure in bar that the actuator may apply.
BrakeActuatorServiceIf	RunActuatorSelfTest	out SelfTestResultType TestResult	Std_ReturnType	Runs the actuator self-test and returns PASSED, FAILED or NOT_COMPLETED.
BrakeControlInfoIf	GetCalibrationSetId	out uint8 CalibSetId	Std_ReturnType	Returns the identifier of the active calibration set.
BrakeControlInfoIf	GetSlipThreshold	out float32 SlipThresholdPct	Std_ReturnType	Returns the active wheel slip threshold in percent.

4.5 Data Types

Table 25: Implementation data types

Data type	Base type	Definition
float32	IEEE 754 single precision	Used for all physical values (wheel speed, pressure, command, temperature, speed).
uint8	Unsigned 8 bit	Used for identifiers and enumerations.
uint16	Unsigned 16 bit	Used for EventId.
boolean	Boolean	FALSE (0) or TRUE (1); used for ABS_Active and DegradedModeRequest.
Std_ReturnType	uint8	E_OK (0) or E_NOT_OK (1); return type of all client-server operations.
VehicleModeType	uint8 enumeration	VEH_MODE_OFF (0), VEH_MODE_STANDBY (1), VEH_MODE_DRIVE (2), VEH_MODE_SERVICE (3).
ActuatorStatusType	uint8 enumeration	ACT_OK (0), ACT_BUSY (1), ACT_DEGRADED (2), ACT_FAULT (3).
SelfTestResultType	uint8 enumeration	TEST_PASSED (0), TEST_FAILED (1), TEST_NOT_COMPLETED (2).
EventStatusType	uint8 enumeration	EVENT_PASSED (0), EVENT_FAILED (1).

4.6 Interface Classification Summary

Sender-Receiver interfaces (7): WheelSpeedIf, BrakePressureIf, BrakeCommandIf, ABSStatusIf, VehicleStateIf, BrakeTemperatureIf, DiagStatusIf.

Client-Server interfaces (3): DiagnosticEventIf, BrakeActuatorServiceIf, BrakeControlInfoIf.

5. Ports

5.1 Port Concepts

Each SW-C owns provided ports (P-ports) and required ports (R-ports). A provided port of a Sender-Receiver interface is a sender; a required port is a receiver. A provided port of a Client-Server interface is a server; a required port is a client. Port names are unique within the owning SW-C. The port name RP_DiagEvent therefore appears once for each client SW-C.

5.2 Port Definitions

Table 26: Port definitions

ID	Port name	Direction	Owning SW-C	Interface reference	Interface type	Role
P-01	PP_BrakeCommand	Provided	BrakeControlSWC	BrakeCommandIf	Sender-Receiver	Sender
P-02	PP_ABSStatus	Provided	BrakeControlSWC	ABSStatusIf	Sender-Receiver	Sender
P-03	PP_BrakeControlInfo	Provided	BrakeControlSWC	BrakeControlInfoIf	Client-Server	Server
P-04	RP_WheelSpeed	Required	BrakeControlSWC	WheelSpeedIf	Sender-Receiver	Receiver
P-05	RP_BrakePressure	Required	BrakeControlSWC	BrakePressureIf	Sender-Receiver	Receiver
P-06	RP_VehicleState	Required	BrakeControlSWC	VehicleStateIf	Sender-Receiver	Receiver
P-07	RP_BrakeTemperature	Required	BrakeControlSWC	BrakeTemperatureIf	Sender-Receiver	Receiver
P-08	RP_DiagStatus	Required	BrakeControlSWC	DiagStatusIf	Sender-Receiver	Receiver
P-09	RP_BrakeActuatorService	Required	BrakeControlSWC	BrakeActuatorServiceIf	Client-Server	Client
P-10	RP_DiagEvent	Required	BrakeControlSWC	DiagnosticEventIf	Client-Server	Client
P-11	PP_WheelSpeed	Provided	WheelSpeedSensorSWC	WheelSpeedIf	Sender-Receiver	Sender
P-12	RP_DiagEvent	Required	WheelSpeedSensorSWC	DiagnosticEventIf	Client-Server	Client
P-13	PP_BrakePressure	Provided	BrakePressureSensorSWC	BrakePressureIf	Sender-Receiver	Sender
P-14	RP_DiagEvent	Required	BrakePressureSensorSWC	DiagnosticEventIf	Client-Server	Client
P-15	PP_BrakeTemperature	Provided	BrakeActuatorSWC	BrakeTemperatureIf	Sender-Receiver	Sender
P-16	PP_BrakeActuatorService	Provided	BrakeActuatorSWC	BrakeActuatorServiceIf	Client-Server	Server
P-17	RP_BrakeCommand	Required	BrakeActuatorSWC	BrakeCommandIf	Sender-Receiver	Receiver
P-18	RP_DiagEvent	Required	BrakeActuatorSWC	DiagnosticEventIf	Client-Server	Client
P-19	PP_VehicleState	Provided	VehicleStateManagerSWC	VehicleStateIf	Sender-Receiver	Sender
P-20	RP_WheelSpeed	Required	VehicleStateManagerSWC	WheelSpeedIf	Sender-Receiver	Receiver
P-21	RP_ABSStatus	Required	VehicleStateManagerSWC	ABSStatusIf	Sender-Receiver	Receiver
P-22	RP_DiagEvent	Required	VehicleStateManagerSWC	DiagnosticEventIf	Client-Server	Client
P-23	PP_DiagEvent	Provided	DiagnosticManagerSWC	DiagnosticEventIf	Client-Server	Server
P-24	PP_DiagStatus	Provided	DiagnosticManagerSWC	DiagStatusIf	Sender-Receiver	Sender
P-25	RP_BrakeControlInfo	Required	DiagnosticManagerSWC	BrakeControlInfoIf	Client-Server	Client
P-26	RP_BrakeActuatorService	Required	DiagnosticManagerSWC	BrakeActuatorServiceIf	Client-Server	Client
P-27	RP_ABSStatus	Required	DiagnosticManagerSWC	ABSStatusIf	Sender-Receiver	Receiver
P-28	RP_VehicleState	Required	DiagnosticManagerSWC	VehicleStateIf	Sender-Receiver	Receiver
P-29	RP_BrakeTemperature	Required	DiagnosticManagerSWC	BrakeTemperatureIf	Sender-Receiver	Receiver

5.3 Port Exposure Summary

Table 27: Ports per SW-C

SW-C	Provided ports	Required ports
BrakeControlSWC	PP_BrakeCommand, PP_ABSStatus, PP_BrakeControlInfo	RP_WheelSpeed, RP_BrakePressure, RP_VehicleState, RP_BrakeTemperature, RP_DiagStatus, RP_BrakeActuatorService, RP_DiagEvent
WheelSpeedSensorSWC	PP_WheelSpeed	RP_DiagEvent
BrakePressureSensorSWC	PP_BrakePressure	RP_DiagEvent
BrakeActuatorSWC	PP_BrakeTemperature, PP_BrakeActuatorService	RP_BrakeCommand, RP_DiagEvent
VehicleStateManagerSWC	PP_VehicleState	RP_WheelSpeed, RP_ABSStatus, RP_DiagEvent
DiagnosticManagerSWC	PP_DiagEvent, PP_DiagStatus	RP_BrakeControlInfo, RP_BrakeActuatorService, RP_ABSStatus, RP_VehicleState, RP_BrakeTemperature

6. Signals

6.1 Signal Overview

VBCS exchanges 11 signals. Each signal is a data element of a Sender-Receiver interface. Signals exchanged between SW-Cs on different ECUs are mapped to CAN messages (Section 7). Signals exchanged between SW-Cs on the same ECU use the RTE only.

6.2 Signal Definitions

Table 28: Signal definitions

Signal name	Data type	Unit	Description	Source SW-C	Destination SW-C	Bus mapping
WheelSpeed_FL	float32	km/h	Wheel speed of the front-left wheel.	WheelSpeedSensorSWC	BrakeControlSWC VehicleStateManagerSWC	CAN: WHEEL_SPEED_01 (0x1A0)
WheelSpeed_FR	float32	km/h	Wheel speed of the front-right wheel.	WheelSpeedSensorSWC	BrakeControlSWC VehicleStateManagerSWC	CAN: WHEEL_SPEED_01 (0x1A0)
WheelSpeed_RL	float32	km/h	Wheel speed of the rear-left wheel.	WheelSpeedSensorSWC	BrakeControlSWC VehicleStateManagerSWC	CAN: WHEEL_SPEED_01 (0x1A0)
WheelSpeed_RR	float32	km/h	Wheel speed of the rear-right wheel.	WheelSpeedSensorSWC	BrakeControlSWC VehicleStateManagerSWC	CAN: WHEEL_SPEED_01 (0x1A0)
BrakePressure	float32	bar	Hydraulic brake pressure representing the driver brake demand.	BrakePressureSensorSWC	BrakeControlSWC	CAN: BRAKE_PRESSURE_01 (0x1B0)
BrakeCommand	float32	%	Brake actuator command, 0 to 100 percent of maximum actuation.	BrakeControlSWC	BrakeActuatorSWC	CAN: BRAKE_CMD_01 (0x2A0)
ABS_Active	boolean	-	TRUE while ABS modulation is active.	BrakeControlSWC	VehicleStateManagerSWC DiagnosticManagerSWC	RTE (BCU-internal); CAN broadcast: ABS_STATUS_01 (0x2B0)
VehicleSpeed	float32	km/h	Estimated vehicle reference speed.	VehicleStateManagerSWC	BrakeControlSWC DiagnosticManagerSWC	RTE (BCU-internal); CAN broadcast: VEHICLE_STATE_01 (0x3A0)
VehicleMode	VehicleMode Type (uint8)	-	Current vehicle mode (enumeration).	VehicleStateManagerSWC	BrakeControlSWC DiagnosticManagerSWC	RTE (BCU-internal); CAN broadcast: VEHICLE_STATE_01 (0x3A0)
BrakeTemperature	float32	degC	Measured brake disc temperature.	BrakeActuatorSWC	BrakeControlSWC DiagnosticManagerSWC	CAN: BRAKE_TEMP_01 (0x1C0)
DegradedModeRequest	boolean	-	TRUE when the diagnostic manager requests degraded mode.	DiagnosticManagerSWC	BrakeControlSWC	None (RTE only, not mapped to CAN)

6.3 Signal-to-Interface Mapping

Table 29: Signal to interface and port mapping

Signal	Interface	Provider port	Receiver ports
WheelSpeed_FL	WheelSpeedIf	WheelSpeedSensorSWC.PP_WheelSpeed	BrakeControlSWC.RP_WheelSpeed VehicleStateManagerSWC.RP_WheelSpeed
WheelSpeed_FR	WheelSpeedIf	WheelSpeedSensorSWC.PP_WheelSpeed	BrakeControlSWC.RP_WheelSpeed VehicleStateManagerSWC.RP_WheelSpeed
WheelSpeed_RL	WheelSpeedIf	WheelSpeedSensorSWC.PP_WheelSpeed	BrakeControlSWC.RP_WheelSpeed VehicleStateManagerSWC.RP_WheelSpeed
WheelSpeed_RR	WheelSpeedIf	WheelSpeedSensorSWC.PP_WheelSpeed	BrakeControlSWC.RP_WheelSpeed VehicleStateManagerSWC.RP_WheelSpeed
BrakePressure	BrakePressureIf	BrakePressureSensorSWC.PP_BrakePressure	BrakeControlSWC.RP_BrakePressure
BrakeCommand	BrakeCommandIf	BrakeControlSWC.PP_BrakeCommand	BrakeActuatorSWC.RP_BrakeCommand
ABS_Active	ABSStatusIf	BrakeControlSWC.PP_ABSStatus	VehicleStateManagerSWC.RP_ABSStatus DiagnosticManagerSWC.RP_ABSStatus
VehicleSpeed	VehicleStateIf	VehicleStateManagerSWC.PP_VehicleState	BrakeControlSWC.RP_VehicleState DiagnosticManagerSWC.RP_VehicleState
VehicleMode	VehicleStateIf	VehicleStateManagerSWC.PP_VehicleState	BrakeControlSWC.RP_VehicleState DiagnosticManagerSWC.RP_VehicleState
BrakeTemperature	BrakeTemperatureIf	BrakeActuatorSWC.PP_BrakeTemperature	BrakeControlSWC.RP_BrakeTemperature DiagnosticManagerSWC.RP_BrakeTemperature
DegradedModeRequest	DiagStatusIf	DiagnosticManagerSWC.PP_DiagStatus	BrakeControlSWC.RP_DiagStatus

6.4 Signal Producers and Consumers

WheelSpeedSensorSWC transmits: WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL and WheelSpeed_RR. These signals are received by BrakeControlSWC and VehicleStateManagerSWC.

BrakePressure is a float32 signal in bar. It is transmitted by BrakePressureSensorSWC and received by BrakeControlSWC.

WheelSpeed_FL has the data type float32 and the unit km/h. On CAN it is carried as a 12-bit unsigned raw value with a factor of 0.1 (see Section 7.3).

BrakeCommand is transmitted by BrakeControlSWC and received by BrakeActuatorSWC. **BrakeTemperature** is transmitted by BrakeActuatorSWC and received by BrakeControlSWC and DiagnosticManagerSWC. **VehicleSpeed** is transmitted by VehicleStateManagerSWC and received by BrakeControlSWC and DiagnosticManagerSWC. **ABS_Active** is transmitted by BrakeControlSWC and received by VehicleStateManagerSWC and DiagnosticManagerSWC.

7. CAN Communication

7.1 Network Description

The CAN network VBC_CAN_A is a Classical CAN network with a bit rate of 500 kbit/s and 11-bit identifiers. A lower CAN ID has a higher priority. All VBCS messages use the Intel (little-endian) byte order. The estimated worst-case bus load, including bit stuffing, is approximately 9 percent.

7.2 CAN Message Mapping

Table 30: CAN message mappings

Message name	CAN ID	DLC	Cycle	Sender ECU	Receiver	Signals carried
WHEEL_SPEED_01	0x1A0	8	10 ms	SIE	BCU	WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL, WheelSpeed_RR
BRAKE_PRESSURE_01	0x1B0	4	10 ms	SIE	BCU	BrakePressure
BRAKE_TEMP_01	0x1C0	4	50 ms	BAM	BCU	BrakeTemperature
BRAKE_CMD_01	0x2A0	4	10 ms	BCU	BAM	BrakeCommand
ABS_STATUS_01	0x2B0	3	20 ms	BCU	Broadcast	ABS_Active
VEHICLE_STATE_01	0x3A0	4	20 ms	BCU	Broadcast	VehicleSpeed, VehicleMode

The CAN message that carries **BrakeCommand** is **BRAKE_CMD_01** (CAN ID 0x2A0). It is transmitted by the BCU every 10 ms and received by the BAM. The CAN message that carries **BrakePressure** is BRAKE_PRESSURE_01 (CAN ID 0x1B0). The four wheel speeds are carried by WHEEL_SPEED_01 (CAN ID 0x1A0), BrakeTemperature by BRAKE_TEMP_01 (CAN ID 0x1C0), ABS_Active by ABS_STATUS_01 (CAN ID 0x2B0), and VehicleSpeed and VehicleMode by VEHICLE_STATE_01 (CAN ID 0x3A0).

Table 31: CAN message to SW-C mapping

Message name	Source SW-C	Destination SW-C (via RTE)
WHEEL_SPEED_01	WheelSpeedSensorSWC	BrakeControlSWC VehicleStateManagerSWC
BRAKE_PRESSURE_01	BrakePressureSensorSWC	BrakeControlSWC
BRAKE_TEMP_01	BrakeActuatorSWC	BrakeControlSWC DiagnosticManagerSWC
BRAKE_CMD_01	BrakeControlSWC	BrakeActuatorSWC
ABS_STATUS_01	BrakeControlSWC	VehicleStateManagerSWC DiagnosticManagerSWC (BCU-internal through RTE; frame also broadcast)
VEHICLE_STATE_01	VehicleStateManagerSWC	BrakeControlSWC DiagnosticManagerSWC (BCU-internal through RTE; frame also broadcast)

7.3 Signal Layout

Table 32: CAN signal layout (Intel byte order)

Message	Signal or field	Start bit	Length	Raw type	Factor	Offset	Physical range
WHEEL_SPEED_01	WheelSpeed_FL	0	12	uint16	0.1	0	0 to 409.5 km/h
WHEEL_SPEED_01	WheelSpeed_FR	12	12	uint16	0.1	0	0 to 409.5 km/h
WHEEL_SPEED_01	WheelSpeed_RL	24	12	uint16	0.1	0	0 to 409.5 km/h
WHEEL_SPEED_01	WheelSpeed_RR	36	12	uint16	0.1	0	0 to 409.5 km/h
WHEEL_SPEED_01	WS01_Counter (E2E)	48	4	uint8	1	0	0 to 15
WHEEL_SPEED_01	WS01_CRC (E2E)	56	8	uint8	1	0	0 to 255
BRAKE_PRESSURE_01	BrakePressure	0	12	uint16	0.1	0	0 to 409.5 bar
BRAKE_PRESSURE_01	BP01_Counter (E2E)	16	4	uint8	1	0	0 to 15
BRAKE_PRESSURE_01	BP01_CRC (E2E)	24	8	uint8	1	0	0 to 255
BRAKE_TEMP_01	BrakeTemperature	0	12	uint16	0.25	-40	-40 to 983.75 degC
BRAKE_TEMP_01	BT01_Counter (E2E)	16	4	uint8	1	0	0 to 15
BRAKE_TEMP_01	BT01_CRC (E2E)	24	8	uint8	1	0	0 to 255
BRAKE_CMD_01	BrakeCommand	0	10	uint16	0.1	0	0 to 102.3 %
BRAKE_CMD_01	BC01_Counter (E2E)	16	4	uint8	1	0	0 to 15
BRAKE_CMD_01	BC01_CRC (E2E)	24	8	uint8	1	0	0 to 255
ABS_STATUS_01	ABS_Active	0	1	boolean	1	0	0 or 1
ABS_STATUS_01	AS01_Counter (E2E)	8	4	uint8	1	0	0 to 15
ABS_STATUS_01	AS01_CRC (E2E)	16	8	uint8	1	0	0 to 255
VEHICLE_STATE_01	VehicleSpeed	0	12	uint16	0.1	0	0 to 409.5 km/h
VEHICLE_STATE_01	VehicleMode	12	4	uint8	1	0	0 to 3 (enumeration)
VEHICLE_STATE_01	VS01_Counter (E2E)	16	4	uint8	1	0	0 to 15
VEHICLE_STATE_01	VS01_CRC (E2E)	24	8	uint8	1	0	0 to 255

7.4 Timing and Protection

Every CAN message contains a 4-bit alive counter and an 8-bit CRC (E2E fields in Table 32). A receiver treats a message as invalid when the counter does not increment by one modulo 16 or when the CRC check fails. Reception timeouts are monitored by the Com module of the receiving ECU.

Table 33: Reception timeout monitoring

Message	Nominal cycle	Timeout	Monitoring SW-C	Related error
WHEEL_SPEED_01	10 ms	30 ms	BrakeControlSWC, VehicleStateManagerSWC	ERR-01
BRAKE_PRESSURE_01	10 ms	30 ms	BrakeControlSWC	ERR-02
BRAKE_CMD_01	10 ms	30 ms	BrakeActuatorSWC	ERR-03
BRAKE_TEMP_01	50 ms	250 ms	BrakeControlSWC, DiagnosticManagerSWC	BrakeTemperature treated as invalid; no fade compensation

8. Functional Flows

8.1 Flow Overview

Table 34: Functional flow overview

Flow ID	Flow name	Participating SW-Cs	Trigger	Flow chain
FF-01	Primary braking flow	WheelSpeedSensorSWC BrakePressureSensorSWC VehicleStateManagerSWC BrakeControlSWC BrakeActuatorSWC	Driver brake demand or wheel speed change	WheelSpeedSensorSWC → BrakeControlSWC → BrakeActuatorSWC and BrakePressureSensorSWC → BrakeControlSWC → BrakeActuatorSWC
FF-02	Wheel-speed to brake-command flow	WheelSpeedSensorSWC BrakeControlSWC BrakeActuatorSWC	Every 10 ms	WheelSpeedSensorSWC → BrakeControlSWC → BrakeActuatorSWC
FF-03	Brake-pressure to brake-command flow	BrakePressureSensorSWC BrakeControlSWC BrakeActuatorSWC	Every 10 ms	BrakePressureSensorSWC → BrakeControlSWC → BrakeActuatorSWC
FF-04	Vehicle speed estimation flow	WheelSpeedSensorSWC VehicleStateManagerSWC BrakeControlSWC	Every 20 ms	WheelSpeedSensorSWC → VehicleStateManagerSWC → BrakeControlSWC
FF-05	ABS status flow	BrakeControlSWC VehicleStateManagerSWC DiagnosticManagerSWC	Slip above threshold	BrakeControlSWC → VehicleStateManagerSWC and BrakeControlSWC → DiagnosticManagerSWC
FF-06	Brake temperature and degraded mode flow	BrakeActuatorSWC BrakeControlSWC DiagnosticManagerSWC	Every 50 ms	BrakeActuatorSWC → DiagnosticManagerSWC → BrakeControlSWC
FF-07	Diagnostic event reporting flow	WheelSpeedSensorSWC BrakePressureSensorSWC BrakeActuatorSWC VehicleStateManagerSWC BrakeControlSWC DiagnosticManagerSWC	Fault detected	Reporting SW-C → DiagnosticManagerSWC
FF-08	Actuator service flow	BrakeControlSWC DiagnosticManagerSWC BrakeActuatorSWC	Every 10 ms and at standby	BrakeControlSWC → BrakeActuatorSWC and DiagnosticManagerSWC → BrakeActuatorSWC

SW-Cs participating in the braking functional flow (FF-01): WheelSpeedSensorSWC, BrakePressureSensorSWC, VehicleStateManagerSWC, BrakeControlSWC and BrakeActuatorSWC. DiagnosticManagerSWC does not participate in the nominal braking flow; it supervises the system in flows FF-05, FF-06 and FF-07.

8.2 FF-01 Primary Braking Flow

The primary braking flow combines two input paths that meet in BrakeControlSWC and produce one output to BrakeActuatorSWC. Path A is WheelSpeedSensorSWC → BrakeControlSWC → BrakeActuatorSWC (flow FF-02). Path B is BrakePressureSensorSWC → BrakeControlSWC → BrakeActuatorSWC (flow FF-03). VehicleStateManagerSWC supplies VehicleSpeed to BrakeControlSWC (flow FF-04). Participating SW-Cs: WheelSpeedSensorSWC, BrakePressureSensorSWC, VehicleStateManagerSWC, BrakeControlSWC and BrakeActuatorSWC.

Table 35: Steps of flow FF-01

Step	From	Action	Interface or signal	To
1	WheelSpeedSensorSWC	Provides the four wheel speeds.	WheelSpeedIf	BrakeControlSWC
2	BrakePressureSensorSWC	Provides BrakePressure.	BrakePressureIf	BrakeControlSWC
3	VehicleStateManagerSWC	Provides VehicleSpeed.	VehicleStateIf	BrakeControlSWC
4	BrakeControlSWC	Calculates BrakeCommand in BrakeControl_Run10ms.	BrakeCommandIf	BrakeActuatorSWC
5	BrakeActuatorSWC	Applies the hydraulic actuation and publishes BrakeTemperature.	BrakeTemperatureIf	BrakeControlSWC

8.3 FF-02 Wheel-Speed to Brake-Command Flow

Chain: WheelSpeedSensorSWC → BrakeControlSWC → BrakeActuatorSWC. The flow converts measured wheel speeds into a brake command.

Table 36: Steps of flow FF-02

Step	From	Action	Interface or signal	To
1	WheelSpeedSensorSWC	WheelSpeed_Run10ms acquires, filters and validates the four wheel speeds.	WheelSpeed_FL to WheelSpeed_RR	PP_WheelSpeed
2	WheelSpeedSensorSWC	Writes the wheel speeds to PP_WheelSpeed; Com on the SIE packs them into WHEEL_SPEED_01 (0x1A0).	WheelSpeedIf	CAN VBC_CAN_A
3	CAN VBC_CAN_A	Com on the BCU unpacks WHEEL_SPEED_01 and performs the E2E check.	WHEEL_SPEED_01	RP_WheelSpeed
4	BrakeControlSWC	BrakeControl_Run10ms reads the wheel speeds through RP_WheelSpeed.	WheelSpeedIf	BrakeControlSWC
5	BrakeControlSWC	Computes wheel slip against VehicleSpeed; if slip is above 15 percent, ABS modulation starts and ABS_Active is set.	ABS_Active	PP_ABSStatus
6	BrakeControlSWC	Writes BrakeCommand to PP_BrakeCommand; Com packs it into BRAKE_CMD_01 (0x2A0).	BrakeCommandIf	CAN VBC_CAN_A
7	BrakeActuatorSWC	BrakeActuator_Run10ms reads BrakeCommand through RP_BrakeCommand and drives the actuator.	BrakeCommandIf	Hydraulic actuator

8.4 FF-03 Brake-Pressure to Brake-Command Flow

Chain: BrakePressureSensorSWC → BrakeControlSWC → BrakeActuatorSWC. The flow converts the driver brake demand into a brake command.

Table 37: Steps of flow FF-03

Step	From	Action	Interface or signal	To
1	BrakePressureSensorSWC	BrakePressure_Run10ms samples the transducer and validates the range 0 to 250 bar.	BrakePressure	PP_BrakePressure
2	BrakePressureSensorSWC	Com on the SIE packs BrakePressure into BRAKE_PRESSURE_01 (0x1B0).	BrakePressureIf	CAN VBC_CAN_A
3	CAN VBC_CAN_A	Com on the BCU unpacks BRAKE_PRESSURE_01 and performs the E2E check.	BRAKE_PRESSURE_01	RP_BrakePressure
4	BrakeControlSWC	Reads BrakePressure through RP_BrakePressure and interprets it as the driver brake demand.	BrakePressureIf	BrakeControlSWC
5	BrakeControlSWC	Maps the demand to a BrakeCommand percentage and applies brake-fade compensation using BrakeTemperature.	BrakeCommand	PP_BrakeCommand
6	BrakeControlSWC	Com packs BrakeCommand into BRAKE_CMD_01 (0x2A0).	BrakeCommandIf	CAN VBC_CAN_A
7	BrakeActuatorSWC	Receives BrakeCommand through RP_BrakeCommand and applies the pressure.	BrakeCommandIf	Hydraulic actuator

8.5 FF-04 Vehicle Speed Estimation Flow

Chain: WheelSpeedSensorSWC → VehicleStateManagerSWC → BrakeControlSWC. DiagnosticManagerSWC also receives VehicleSpeed.

Table 38: Steps of flow FF-04

Step	From	Action	Interface or signal	To
1	WheelSpeedSensorSWC	Provides the four wheel speeds in WHEEL_SPEED_01.	WheelSpeedIf	VehicleStateManagerSWC
2	VehicleStateManagerSWC	VehicleState_Run20ms reads the wheel speeds through RP_WheelSpeed.	WheelSpeedIf	VehicleStateManagerSWC
3	VehicleStateManagerSWC	Estimates VehicleSpeed and derives VehicleMode.	VehicleSpeed, VehicleMode	PP_VehicleState
4	VehicleStateManagerSWC	Provides VehicleStateIf to BrakeControlSWC and DiagnosticManagerSWC.	VehicleStateIf	BrakeControlSWC, DiagnosticManagerSWC

8.6 FF-05 ABS Status Flow

Chain: BrakeControlSWC → VehicleStateManagerSWC and BrakeControlSWC → DiagnosticManagerSWC.

Table 39: Steps of flow FF-05

Step	From	Action	Interface or signal	To
1	BrakeControlSWC	Detects slip above the threshold and sets ABS_Active to TRUE.	ABS_Active	PP_ABSStatus
2	BrakeControlSWC	Provides ABSStatusIf to the receivers; Com also broadcasts ABS_STATUS_01 (0x2B0).	ABSStatusIf	VehicleStateManagerSWC, DiagnosticManagerSWC
3	VehicleStateManagerSWC	Holds the reference speed estimate while ABS_Active is TRUE.	ABS_Active	PP_VehicleState
4	DiagnosticManagerSWC	Measures the duration of ABS_Active; above 30 s it raises EVT_ABS_StuckActive.	ABS_Active	Dem

8.7 FF-06 Brake Temperature and Degraded Mode Flow

Chain: BrakeActuatorSWC → DiagnosticManagerSWC → BrakeControlSWC. BrakeControlSWC also receives BrakeTemperature directly.

Table 40: Steps of flow FF-06

Step	From	Action	Interface or signal	To
1	BrakeActuatorSWC	BrakeActuator_TempRun50ms measures the disc temperature and writes BrakeTemperature.	BrakeTemperature	PP_BrakeTemperature
2	BrakeActuatorSWC	Com on the BAM packs BrakeTemperature into BRAKE_TEMP_01 (0x1C0).	BrakeTemperatureIf	CAN VBC_CAN_A
3	DiagnosticManagerSWC	DiagMgr_Run20ms reads BrakeTemperature through RP_BrakeTemperature.	BrakeTemperatureIf	DiagnosticManagerSWC
4	DiagnosticManagerSWC	If BrakeTemperature is above 600 degC for 2 s, raises EVT_BRAKE_OverTemp and sets DegradedModeRequest to TRUE.	DegradedModeRequest	PP_DiagStatus
5	BrakeControlSWC	Reads DegradedModeRequest through RP_DiagStatus and enters degraded mode.	DiagStatusIf	BrakeControlSWC
6	BrakeControlSWC	Calls SetActuatorPressureLimit with 120 bar on the BrakeActuatorSWC server.	BrakeActuatorServiceIf	BrakeActuatorSWC

8.8 FF-07 Diagnostic Event Reporting Flow

Chain: reporting SW-C → DiagnosticManagerSWC. WheelSpeedSensorSWC, BrakePressureSensorSWC, BrakeActuatorSWC, VehicleStateManagerSWC and BrakeControlSWC are clients of DiagnosticEventIf.

Table 41: Steps of flow FF-07

Step	From	Action	Interface or signal	To
1	Reporting SW-C	Detects a fault condition listed in Table 46.	EventId	RP_DiagEvent
2	Reporting SW-C	Calls ReportDiagEvent with EventId and EventStatus.	DiagnosticEventIf	DiagnosticManagerSWC
3	DiagnosticManagerSWC	DiagMgr_OnReportEvent debounces the event and forwards it to the Dem.	ReportDiagEvent	Dem
4	DiagnosticManagerSWC	Evaluates whether DegradedModeRequest must be set.	DegradedModeRequest	PP_DiagStatus

8.9 FF-08 Actuator Service Flow

Chain: BrakeControlSWC → BrakeActuatorSWC and DiagnosticManagerSWC → BrakeActuatorSWC. Both clients use BrakeActuatorServiceIf.

Table 42: Steps of flow FF-08

Step	From	Action	Interface or signal	To
1	BrakeControlSWC	Calls GetActuatorStatus each control cycle through RP_BrakeActuatorService.	BrakeActuatorServiceIf	BrakeActuatorSWC
2	BrakeControlSWC	In degraded mode, calls SetActuatorPressureLimit with LimitBar equal to 120.	BrakeActuatorServiceIf	BrakeActuatorSWC
3	DiagnosticManagerSWC	When VehicleMode is VEH_MODE_STANDBY, calls RunActuatorSelfTest.	BrakeActuatorServiceIf	BrakeActuatorSWC
4	BrakeActuatorSWC	BrakeActuator_OnServiceRequest executes the operation and returns Std_ReturnType.	BrakeActuatorServiceIf	Calling SW-C

9. SW-C Dependencies

9.1 Dependency Table

Each dependency is expressed as Source SW-C → Interface → Target SW-C. For Sender-Receiver interfaces the source is the sender and the target is the receiver. For Client-Server interfaces the source is the client and the target is the server. In the Kind column, SR means Sender-Receiver data flow and CS means Client-Server operation call. The 19 dependencies below are derived from the port definitions in Section 5.

Table 43: SW-C dependencies

ID	Source SW-C	Interface	Target SW-C	Kind	Source port	Target port
D-01	WheelSpeedSensorSWC	WheelSpeedIf	BrakeControlSWC	SR data flow	PP_WheelSpeed	RP_WheelSpeed
D-02	WheelSpeedSensorSWC	WheelSpeedIf	VehicleStateManagerSWC	SR data flow	PP_WheelSpeed	RP_WheelSpeed
D-03	BrakePressureSensorSWC	BrakePressureIf	BrakeControlSWC	SR data flow	PP_BrakePressure	RP_BrakePressure
D-04	BrakeControlSWC	BrakeCommandIf	BrakeActuatorSWC	SR data flow	PP_BrakeCommand	RP_BrakeCommand
D-05	BrakeControlSWC	ABSStatusIf	VehicleStateManagerSWC	SR data flow	PP_ABSStatus	RP_ABSStatus
D-06	BrakeControlSWC	ABSStatusIf	DiagnosticManagerSWC	SR data flow	PP_ABSStatus	RP_ABSStatus
D-07	VehicleStateManagerSWC	VehicleStateIf	BrakeControlSWC	SR data flow	PP_VehicleState	RP_VehicleState
D-08	VehicleStateManagerSWC	VehicleStateIf	DiagnosticManagerSWC	SR data flow	PP_VehicleState	RP_VehicleState
D-09	BrakeActuatorSWC	BrakeTemperatureIf	BrakeControlSWC	SR data flow	PP_BrakeTemperature	RP_BrakeTemperature
D-10	BrakeActuatorSWC	BrakeTemperatureIf	DiagnosticManagerSWC	SR data flow	PP_BrakeTemperature	RP_BrakeTemperature
D-11	DiagnosticManagerSWC	DiagStatusIf	BrakeControlSWC	SR data flow	PP_DiagStatus	RP_DiagStatus
D-12	BrakeControlSWC	DiagnosticEventIf	DiagnosticManagerSWC	CS call	RP_DiagEvent	PP_DiagEvent
D-13	WheelSpeedSensorSWC	DiagnosticEventIf	DiagnosticManagerSWC	CS call	RP_DiagEvent	PP_DiagEvent
D-14	BrakePressureSensorSWC	DiagnosticEventIf	DiagnosticManagerSWC	CS call	RP_DiagEvent	PP_DiagEvent
D-15	BrakeActuatorSWC	DiagnosticEventIf	DiagnosticManagerSWC	CS call	RP_DiagEvent	PP_DiagEvent
D-16	VehicleStateManagerSWC	DiagnosticEventIf	DiagnosticManagerSWC	CS call	RP_DiagEvent	PP_DiagEvent
D-17	BrakeControlSWC	BrakeActuatorServiceIf	BrakeActuatorSWC	CS call	RP_BrakeActuatorService	PP_BrakeActuatorService
D-18	DiagnosticManagerSWC	BrakeActuatorServiceIf	BrakeActuatorSWC	CS call	RP_BrakeActuatorService	PP_BrakeActuatorService
D-19	DiagnosticManagerSWC	BrakeControlInfoIf	BrakeControlSWC	CS call	RP_BrakeControlInfo	PP_BrakeControlInfo

9.2 Dependencies between BrakeControlSWC and BrakeActuatorSWC

BrakeControlSWC and BrakeActuatorSWC have 3 direct dependencies: D-04: BrakeControlSWC → BrakeCommandIf → BrakeActuatorSWC; D-09: BrakeActuatorSWC → BrakeTemperatureIf → BrakeControlSWC; D-17: BrakeControlSWC → BrakeActuatorServiceIf → BrakeActuatorSWC.

Table 44: Dependencies between BrakeControlSWC and BrakeActuatorSWC

ID	Source SW-C	Interface	Target SW-C	Purpose
D-04	BrakeControlSWC	BrakeCommandIf	BrakeActuatorSWC	BrakeControlSWC sends BrakeCommand to the actuator.
D-09	BrakeActuatorSWC	BrakeTemperatureIf	BrakeControlSWC	BrakeActuatorSWC sends BrakeTemperature to the controller for fade compensation.
D-17	BrakeControlSWC	BrakeActuatorServiceIf	BrakeActuatorSWC	BrakeControlSWC calls GetActuatorStatus and SetActuatorPressureLimit.

9.3 Dependency Matrix

Table 45: Number of dependencies from source SW-C (rows) to target SW-C (columns)

Source / Target	BrakeControl	WheelSpeed	BrakePressure	BrakeActuator	VehicleState	DiagManager
BrakeControlSWC	-	-	-	2	1	2
WheelSpeedSensorSWC	1	-	-	-	1	1
BrakePressureSensorSWC	1	-	-	-	-	1
BrakeActuatorSWC	1	-	-	-	-	2
VehicleStateManagerSWC	1	-	-	-	-	2
DiagnosticManagerSWC	2	-	-	1	-	-

10. Diagnostics

10.1 Diagnostic Architecture

DiagnosticManagerSWC is the central diagnostic component. Every other SW-C reports faults through its required port **RP_DiagEvent**, which is a client port of **DiagnosticEventIf**. DiagnosticManagerSWC provides the server port **PP_DiagEvent**. DiagnosticManagerSWC debounces the events and forwards them to the Diagnostic Event Manager (Dem) in the BCU basic software. Fictional DTC identifiers of the form VBC-xxxx are used.

10.2 Diagnostic Events

Table 46: Diagnostic events

Event ID	Event name	Reporter	Fault condition	Debounce	DTC	Reaction
0x0101	EVT_WSS_FL_Implausible	WheelSpeedSensorSWC	WheelSpeed_FL outside 0 to 350 km/h or gradient above 30 km/h per 10 ms	3 of 5 cycles	VBC-0101	Exclude FL from slip calculation
0x0102	EVT_WSS_FR_Implausible	WheelSpeedSensorSWC	WheelSpeed_FR outside 0 to 350 km/h or gradient above 30 km/h per 10 ms	3 of 5 cycles	VBC-0102	Exclude FR from slip calculation
0x0103	EVT_WSS_RL_Implausible	WheelSpeedSensorSWC	WheelSpeed_RL outside 0 to 350 km/h or gradient above 30 km/h per 10 ms	3 of 5 cycles	VBC-0103	Exclude RL from slip calculation
0x0104	EVT_WSS_RR_Implausible	WheelSpeedSensorSWC	WheelSpeed_RR outside 0 to 350 km/h or gradient above 30 km/h per 10 ms	3 of 5 cycles	VBC-0104	Exclude RR from slip calculation
0x0105	EVT_WSS_Timeout	BrakeControlSWC	WHEEL_SPEED_01 not received within 30 ms	1 occurrence	VBC-0105	Disable ABS; base braking only
0x0201	EVT_BPS_OutOfRange	BrakePressureSensorSWC	BrakePressure outside 0 to 250 bar	3 of 5 cycles	VBC-0201	Hold last valid value for 2 cycles, then degraded mode
0x0202	EVT_BPS_Timeout	BrakeControlSWC	BRAKE_PRESSURE_01 not received within 30 ms	1 occurrence	VBC-0202	Degraded mode
0x0301	EVT_ACT_CommandTimeout	BrakeActuatorSWC	BRAKE_CMD_01 not received within 30 ms	1 occurrence	VBC-0301	Actuator moves to safe release state
0x0302	EVT_ACT_SelfTestFailed	BrakeActuatorSWC	RunActuatorSelfTest returns FAILED	1 occurrence	VBC-0302	Degraded mode
0x0303	EVT_ACT_TempSensorFault	BrakeActuatorSWC	Brake temperature sensor open circuit or short circuit	5 of 10 cycles	VBC-0303	BrakeTemperature set to invalid; no fade compensation
0x0401	EVT_VSM_SpeedPlausibility	VehicleStateManagerSWC	VehicleSpeed deviates more than 20 km/h from all valid wheel speeds	10 cycles	VBC-0401	Reference speed frozen for 200 ms
0x0501	EVT_BRAKE_OverTemp	DiagnosticManagerSWC (internal)	BrakeTemperature above 600 degC for 2 s	2 s	VBC-0501	Request degraded mode
0x0502	EVT_ABS_StuckActive	DiagnosticManagerSWC (internal)	ABS_Active TRUE continuously for more than 30 s	30 s	VBC-0502	Request degraded mode
0x0601	EVT_CTRL_CalibMismatch	BrakeControlSWC	Active calibration set identifier does not match stored set	1 occurrence	VBC-0601	Use default calibration set 0

10.3 Freeze-Frame Data

When an event is confirmed, DiagnosticManagerSWC stores a freeze frame. The freeze frame contains only data available to DiagnosticManagerSWC through its required ports and client calls. BrakePressure is not part of the freeze frame because DiagnosticManagerSWC does not receive it.

Table 47: Freeze-frame data elements

Data element	Source	Access method
VehicleSpeed	VehicleStateManagerSWC	RP_VehicleState
VehicleMode	VehicleStateManagerSWC	RP_VehicleState
BrakeTemperature	BrakeActuatorSWC	RP_BrakeTemperature
ABS_Active	BrakeControlSWC	RP_ABSStatus
CalibSetId	BrakeControlSWC	GetCalibrationSetId through RP_BrakeControlInfo
ActuatorStatus	BrakeActuatorSWC	GetActuatorStatus through RP_BrakeActuatorService

10.4 Diagnostic Self-Test

When VehicleMode equals VEH_MODE_STANDBY, DiagnosticManagerSWC calls RunActuatorSelfTest on BrakeActuatorSWC through BrakeActuatorServiceIf. A result of TEST_FAILED causes BrakeActuatorSWC to report EVT_ACT_SelfTestFailed (0x0302). The self-test is not started while VehicleMode equals VEH_MODE_DRIVE.

11. Error Handling

11.1 Error Handling Strategy

Errors are handled at the lowest possible level. Communication errors are detected by the Com module and by E2E checks, then forwarded to the receiving SW-C through RTE return codes such as RTE_E_MAX_AGE_EXCEEDED. SW-C level errors are reported to DiagnosticManagerSWC using ReportDiagEvent. Client-server calls return Std_ReturnType; E_NOT_OK indicates a failed operation.

11.2 Error Table

Table 48: Error handling

ID	Error	Detected by	Detection condition	Reaction	Recovery
ERR-01	Loss of WHEEL_SPEED_01	BrakeControlSWC	Com timeout of 30 ms or RTE_E_MAX_AGE_EXCEEDED on RP_WheelSpeed	Use last valid wheel speeds for at most 30 ms, then disable ABS and use base braking; report 0x0105	Five consecutive valid frames
ERR-02	Loss of BRAKE_PRESSURE_01	BrakeControlSWC	Com timeout of 30 ms on RP_BrakePressure	Request actuator limit of 120 bar via SetActuatorPressureLimit; report 0x0202	Five consecutive valid frames
ERR-03	Loss of BRAKE_CMD_01	BrakeActuatorSWC	No frame within 30 ms	Move to safe release state; report 0x0301	Five consecutive valid frames
ERR-04	E2E counter or CRC failure	Receiving SW-C	E2E check status not OK for two consecutive frames	Treat the data as invalid and apply the reaction of the corresponding loss error	Two consecutive valid frames
ERR-05	Implausible wheel speed	WheelSpeedSensorSWC	Range or gradient check fails	Mark the wheel speed invalid; slip calculation excludes this wheel; report 0x0101 to 0x0104	Plausible values for 20 cycles
ERR-06	BrakePressure out of range	BrakePressureSensorSWC	Value outside 0 to 250 bar	Hold last valid value for two cycles; report 0x0201	Valid values for 10 cycles
ERR-07	Actuator self-test failed	BrakeActuatorSWC	RunActuatorSelfTest returns FAILED	Report 0x0302; DiagnosticManagerSWC sets DegradedModeRequest to TRUE	Successful self-test at next standby
ERR-08	Brake over-temperature	DiagnosticManagerSWC	BrakeTemperature above 600 degC for 2 s	Report 0x0501 internally; set DegradedModeRequest to TRUE	BrakeTemperature below 500 degC for 10 s
ERR-09	ABS stuck active	DiagnosticManagerSWC	ABS_Active TRUE for more than 30 s	Report 0x0502 internally; set DegradedModeRequest to TRUE	ABS_Active FALSE for 5 s
ERR-10	Operation call failure	Client SW-C	Std_ReturnType E_NOT_OK or RTE_E_TIMEOUT from a client-server call	Retry once in next cycle; after three failures report the related event	First successful call

11.3 Degraded Mode Behavior

In degraded mode, **BrakeControlSWC** limits BrakeCommand to 80 percent and calls SetActuatorPressureLimit with a limit of 120 bar on **BrakeActuatorSWC**. ABS is disabled only if the wheel speed signals are invalid. Degraded mode is left when DegradedModeRequest returns to FALSE; the default pressure limit of 200 bar is then restored.

12. Architecture Summary

12.1 Summary of Architecture Elements

Table 49: Architecture summary

Element	Count	Members
SW-Cs	6	BrakeControlSWC, WheelSpeedSensorSWC, BrakePressureSensorSWC, BrakeActuatorSWC, VehicleStateManagerSWC, DiagnosticManagerSWC
Sender-Receiver interfaces	7	WheelSpeedIf, BrakePressureIf, BrakeCommandIf, ABSStatusIf, VehicleStateIf, BrakeTemperatureIf, DiagStatusIf
Client-Server interfaces	3	DiagnosticEventIf, BrakeActuatorServiceIf, BrakeControlInfoIf
Ports	29	10 provided, 19 required
Signals	11	WheelSpeed_FL, WheelSpeed_FR, WheelSpeed_RL, WheelSpeed_RR, BrakePressure, BrakeCommand, ABS_Active, VehicleSpeed, VehicleMode, BrakeTemperature, DegradedModeRequest
CAN messages	6	WHEEL_SPEED_01, BRAKE_PRESSURE_01, BRAKE_TEMP_01, BRAKE_CMD_01, ABS_STATUS_01, VEHICLE_STATE_01
Dependencies	19	D-01 to D-19
Functional flows	8	FF-01 to FF-08
Diagnostic events	14	0x0101 to 0x0601

12.2 Key Architectural Decisions

- BrakeControlSWC is the only SW-C that calculates BrakeCommand; BrakeActuatorSWC is the only receiver of BrakeCommand.
- BrakePressure is received only by BrakeControlSWC.
- All fault reporting is centralized through DiagnosticEventIf with DiagnosticManagerSWC as the only server.
- DegradedModeRequest is not mapped to CAN because BrakeControlSWC and DiagnosticManagerSWC are both deployed on the BCU.
- ABS_Active, VehicleSpeed and VehicleMode are exchanged through the RTE on the BCU and additionally broadcast on VBC_CAN_A.

12.3 Open Items

- OI-01: Final calibration values for the slip threshold and fade compensation gain are to be defined.
- OI-02: The mapping of diagnostic events to DTC identifiers is preliminary.
- OI-03: Bus load estimation is to be verified with a full network simulation.

End of document VBCHLD-001, revision 1.0.